Performance Testing

Date	9 July 2026
Team ID	xxxxxx
Project Name	**Credit Card Approval Prediction**
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Credit Card Approval Prediction
Type of Testing	Load Testing, Functional Testing
Target Module / API	Credit Card Approval Prediction API (/predict)
Test Environment	Local Machine (Windows, Python 3.x, Flask)
Test Date	7 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Local Machine (Windows, Python 3.x, Flask)	1	30	Prediction generated successfully within acceptable response time
2	Multiple prediction requests	10	60	All requests processed successfully without errors

3	Invalid input testing	5	30	System displays validation/error message
4	Continuous prediction requests	20	120	Stable performance with no crashes or failures

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.78 seconds	Pass	Fast response time
2	Response Time (Max)	< 5 seconds	2.10 seconds	Pass	Within acceptable limit
3	Throughput (Req/sec)	10 Req/sec	14 Req/sec	Pass	Handles multiple requests efficiently
4	Error Rate	< 1%	0%	Pass	No failed requests observed
5	CPU Utilization	< 80%	42%	Pass	CPU usage remained stable
6	Memory Utilization	< 80%	51%	Pass	Memory usage within safe limits

Step 4: Observations & Analysis

Key Findings :-

The application handled multiple prediction requests efficiently with low response time and no significant performance degradation. The machine learning model produced predictions accurately while maintaining stable system performance.

Bottlenecks Identified :-

No major bottlenecks were observed during testing. Minor delays occurred only when processing multiple simultaneous requests, but they remained within acceptable limits.

Optimization Steps Taken :-

- Optimized data preprocessing functions.
- Loaded the trained model only once during application startup.
- Removed redundant computations.

• Improved input validation to reduce processing overhead.

Step 5: Screenshots / Evidence